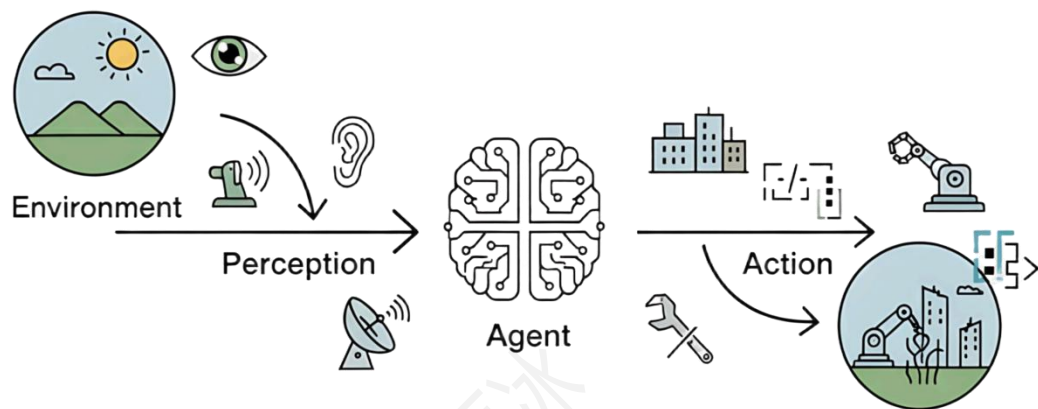


基本概念

基本定义

“Agent”（通常译为“智能体”）指能通过传感器自主感知周围环境，自主做出决策并自主执行相应动作的实体，agent，传感器，执行机构等概念不局限于物理实体，包括软件程序，虚拟角色等实体。



智能体与环境的基本交互循环

自主性体现在能够在**没有人类直接干预**的情况下运作，并对自身行为和内部状态有一定控制能力。

要理解 Agent，关键在于区分它与普通程序（如计算器、播放器）的差异：

普通程序：**被动响应**，只能按照人类预设的固定指令执行（如点“播放”按键才播放音乐）。

Agent：**主动交互**，能主动感知周围环境（如时间、用户行为、设备状态），并根据目标自主调整行为（如感知到你的疲惫自动播放音乐）。

基本组成-感知&决策&执行&通信

感知模块

感知模块是智能体获取环境信息的入口，相当于人类的“感官系统”。其组成结构主要包括：

硬件传感器：如摄像头、麦克风、激光雷达等，用于获取物理环境的信息。

软件接口：如 API、网页爬虫等，用于获取数字环境的信息。

人机交互设备：如键盘、鼠标、触摸屏等，用于接收用户输入。

感知到物理或数字信息后，可以进一步的**数据预处理**，如图像降噪、语音增强等，还可以**整合多模态信息**，处理不同模态信息之间的时间和空间对齐问题，并将其传递给 "大脑"（大模型）进行理解分析。

决策模块

决策模块是智能体的 "大脑"，负责**处理感知信息**、进行**推理和规划**，并**生成行动指令**。其组成结构主要包括：

1.推理与规划引擎：

基于感知信息进行**逻辑推理和不确定性推理**，生成行动计划和策略，实现从目标到行动的转换。

2.学习与优化机制：

通过学习算法不断更新知识库和决策策略，适应环境变化和新的任务需求。

执行模块

执行模块是智能体将决策转化为实际行动的部分，相当于人类的 "运动系统"。其组成结构主要包括：**硬件 Agent**：电机（控制机器人移动）、机械臂（抓取物体）、显示屏（输出信息）；**软件 Agent**：APP 推送通知、文件下载指令、数据库操作。

动作生成与控制：将抽象的决策结果转换为具体的动作序列或控制指令。

工具与接口：提供与外部工具和服务的接口，扩展智能体的能力边界。

执行监控与反馈：监控执行过程，确保动作按计划执行。

通信模块

通信模块是智能体与外部实体（包括其他智能体、用户和系统）进行信息交换的桥梁。Agent 的 “嘴巴和耳朵”，负责与其他 Agent、人类或外部系统交换信息。

常见方式：网络通信（Wi-Fi、蓝牙、5G）、API 接口（软件间数据交互）、自然语言交互（语音、文字）

通信协议与接口：实现不同智能体之间、与外部系统和服务之间的通信协议，如消息传递与解析、远程过程调用、协作和分工等。

与大模型的关系

表 1.1 传统智能体与 LLM 驱动智能体的核心对比

对比维度	传统智能体	LLM驱动的智能体
核心引擎	基于显式编程的逻辑系统	基于预训练模型的推理引擎
知识来源	工程师预定义的规则、算法、知识库	从海量非结构化数据中间接学习内化
处理指令	需结构化、精确的命令	可理解高层级、模糊的自然语言
工作模式	确定性的、可预测的	概率性的、生成式的
泛化/适应性	弱，局限于预设框架	强，具备强大的涌现能力和泛化能力
开发范式	规则设计、算法编程、知识工程	模型训练、提示工程、微调

智能体与大模型的关系仍然从三方面考虑，

感知方面：大模型能够更好地理解感知到的信息，如图像，自然语言，

决策方面：大模型能够帮助智能体进行复杂的推理判断，将复杂任务拆解为一系列子任务，规划下一步的任务；

执行方面：会主动调用外部工具，能够让智能体的动作执行得更加流畅智能；在交互过程中，智能体会将用户的反馈视为新的约束，并据此调整后续的行动

分类与层级体系

智能体可以根据不同标准进行分类，常见的分类方式包括：

按自主性程度分类：

反应式智能体：仅对当前感知信息做出直接反应，没有内部状态或记忆。

认知式智能体：具有内部状态和记忆，能够进行推理和规划，做出更复杂的决策。

完全自主智能体：能够在没有人类干预的情况下自主运行、学习和适应环境。

按体系结构分类：

单体智能体：独立运行的单个智能体系统。

多智能体系统：由多个相互协作的智能体组成的系统，能够共同完成复杂任务。

Hugging Face 的研究团队根据自主性程度对 AI 智能体系统划分成五类层级体系：

Level 0（处理器）：对程序流程没有影响的智能体，如简单的客服聊天机器人。

Level 1（路由器）：决定执行哪些步骤，但需人类确认关键操作。

Level 2（工具调用智能体）：能调用第三方工具，但依赖人类编写的核心逻辑。

Level 3（多步执行智能体）：自主规划任务流程，但在敏感操作时需交出控制权。

Level 4（完全自主智能体）：能够在没有人类约束或监督的情况下编写并执行新代码。

这种层级体系反映了智能体技术从简单到复杂、从辅助到自主的发展路径。

基于时间与反应性的分类

除了内部架构的复杂性，还可以从智能体处理决策的时间维度进行分类。这个视角关注智能体是在接收到信息后立即行动，还是会经过深思熟虑的规划再行动。这揭示了智能体设计中一个核心权衡：追求速度的**反应性（Reactivity）**与追求最优解的**规划性（Deliberation）**之间的平衡，如图 1.3 所示。

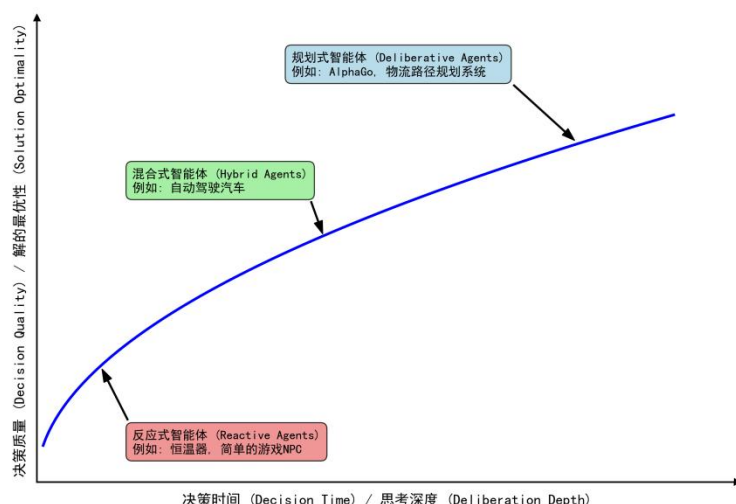


图 1.3 智能体决策时间与质量关系图

反应式智能体 (Reactive Agents) 这类智能体对环境刺激做出近乎即时的响应，决策延迟极低。它们通常遵循从感知到行动的直接映射，不进行或只进行极少的未来规划。上文的**简单反应式**和**基于模型**的智能体都属于此类别。

其核心优势在于**速度快、计算开销低**，这在需要快速决策的动态环境中至关重要。例如，车辆的安全气囊系统必须在碰撞发生的毫秒内做出反应，任何延迟都可能导致严重后果；同样，高频交易机器人也必须依赖反应式决策来捕捉稍纵即逝的市场机会。然而，这种速度的代价是“短视”，由于缺乏长远规划，反应式智能体容易陷入局部最优，难以完成需要多步骤协调的复杂任务。

规划式智能体(Deliberative Agents)

与反应式智能体相对，规划式（或称审议式）智能体在行动前会进行**复杂的思考和规划**。它们不会立即对感知做出反应，而是会先利用其内部的世界模型，系统地探索未来的各种可能性，评估不同行动序列的后果，以期找到一条能够达成目标的**最佳路径**。**基于目标和基于效用的智能体**是典型的规划式智能体。

它们的优势在于决策的战略性和远见。然而，这种优势的另一面是高昂的时间和计算成本。在瞬息万变的环境中，当规划式智能体还在深思熟虑时，采取行动的最佳时机可能早已过去。

混合式智能体(Hybrid Agents)

现实世界的复杂任务，往往既需要即时反应，也需要长远规划。例如，我们之前提到的智能旅行助手，既要能根据用户的即时反馈（如“这家酒店太贵了”）调整推荐（反应性），又要能规划出为期数天的完整旅行方案（规划性）。因此，混合式智能体应运而生，它旨在结合两者的优点，实现反应与规划的平衡。一种经典的混合架构是分层设计：底层是一个快速的反应模块，处理紧急情况和基本动作；高层则是一个审慎的规划模块，负责制定长远目标。而现代的 LLM 智能体，则展现了一种更灵活的混合模式。它们通常在一个“思考-行动-观察”的循环中运作，巧妙地将两种模式融为一体：

规划(Reasoning)：在“思考”阶段，LLM 分析当前状况，规划出下一步的合理行动。这是一个审议过程。

反应(Acting & Observing)：在“行动”和“观察”阶段，智能体与外部工具或环境交互，并立即获得反馈。这是一个反应过程。

通过这种方式，智能体将一个需要长远规划的宏大任务，分解为一系列“规划-反应”的微循环。这使其既能灵活应对环境的即时变化，又能通过连贯的步骤，最终完成复杂的长期目标。

(3) 基于知识表示的分类

这是一个更根本的分类维度，它探究智能体用以决策的知识，究竟是以何种形式存于其“思想”之中。这个问题是人工智能领域一场持续半个多世纪的辩论核心，并塑造了两种截然不同的 AI 文化。

符号主义 AI (Symbolic AI)

符号主义，常被称为传统人工智能，其核心信念是：智能源于对符号的逻辑操作。这里的符号是人类可读的实体（如词语、概念），操作则遵循严格的逻辑规则，如图 1.4 左侧所示。这好比一位一丝不苟的图书管理员，将世界知识整理为清晰的规则库和知识图谱。

其主要优势在于透明和可解释。由于推理步骤明确，其决策过程可以被完整追溯，这在金融、医疗等高风险领域至关重要。然而，其“阿喀琉斯之踵”在于脆弱性：它依赖于一个完备的规则体系，但在充满模糊和例外的现实世界中，任何未被覆盖的新情况都可能导致系统失灵，这就是所谓的“知识获取瓶颈”。

亚符号主义 AI (Sub-symbolic AI)

亚符号主义，或称连接主义，则提供了一幅截然不同的图景。在这里，知识并非显式的规则，而是内隐地分布在一个由大量神经元组成的复杂网络中，是从海量数据中学习到的统计模式。神经网络和深度学习是其代表。

如图 1.4 中间所示，如果说符号主义 AI 是图书管理员，那么亚符号主义 AI 就像一个牙牙学语的孩童。他不是通过学习“猫有四条腿、毛茸茸、会喵喵叫”这样的规则来认识猫的，而是在看过成千上万张猫的图片后，大脑中的神经网络能辨识出“猫”这个概念的视觉模式。这种方法的强大之处在于其模式识别能力和对噪声数据的鲁棒性。它能够轻松处理图像、声音等非结构化数据，这在符号主义 AI 看来是极其困难的任务。

然而，这种强大的直觉能力也伴随着不透明性。亚符号主义系统通常被视为一个黑箱 (Black Box)。它能以惊人的准确率识别出图片中的猫，但你若问它“为什么你认为这是猫？”，它很可能无法给出一个合乎逻辑的解释。此外，它在纯粹的逻辑推理任务上表现不佳，有时会产生看似合理却事实错误的幻觉。

神经符号主义 AI (Neuro-Symbolic AI)

长久以来，符号主义和亚符号主义这两大阵营如同两条平行线，各自发展。为克服上述两种范式的局限，一种“大和解”的思想开始兴起，这就是神经符号主义 AI，也称神经符号混合主义。它的目标，是融合两大范式的优点，创造出一个既能像神经网络一样从数据中学习，又能像符号系统一样进行逻辑推理的混合智能体。它试图弥合感知与认知、直觉与理性之间的鸿沟。诺贝尔经济学奖得主丹尼尔·卡尼曼 (Daniel Kahneman) 在其著作《思考，快与慢》(Thinking, Fast and Slow) 中提出的双系统理论，为我们理解神经符号主义提供了一个绝佳的类比[2]，如图 1.4 所示：

系统 1 是快速、凭直觉、并行的思维模式，类似于亚符号主义 AI 强大的模式识别能力。

系统 2 是缓慢、有条理、基于逻辑的审慎思维，恰如符号主义 AI 的推理过程。



图 1.4 符号主义、亚符号主义与神经符号混合主义的知识表示范式

人类的智能，正源于这两个系统的协同工作。同样，一个真正鲁棒的 AI，也需要兼具二者之长。大语言模型驱动的智能体是神经符号主义的一个极佳实践范例。其内核是一个巨大的神经网络，使其具备模式识别和语言生成能力。然而，当它工作时，它会生成一系列结构化的中间步骤，如思想、计划或 API 调用，这些都是明确的、可操作的符号。通过这种方式，它实现了感知与认知、直觉与理性的初步融合。

Agent 能力演进概述

摘要

本文围绕一个清晰的演进主线展开：Agent 如何从“只能对话的基础聊天模型”，逐步发展为“可调用工具的 Agent”，再进一步发展“多轮有状态、具备记忆与治理能力的 Agent”，以及“具备 Reflect-Observe-Act 风格循环控制、重试控制和安全停止条件的工程化 Agent”。文中使用一组 mock-friendly 的 Python 代码示例。这些示例不依赖真实供应商 SDK，而是通过 `Protocol`、`dataclass` 和可替换接口表达 Agent 的关键演进结构。

背景与研究目标

在很多团队的早期实践里，“Agent”常常被误解为“会聊天的 LLM + 几个工具”。但一旦进入真实工程场景，这种理解很快就不够用了。原因是：

1. 真实任务往往不是单轮完成，而是多轮推进。
2. 真实任务需要显式状态，而不是只靠 messages 隐式推断。
3. 真实任务需要记忆，但更需要记忆管理。
4. 真实任务需要循环，但必须可控、可中断、可回退、可转人工。
5. 真实任务需要测试与评审，因此核心组件必须支持 mock、回放和确定性验证。

Agent 演进总览

阶段	核心能力	主要问题	典型工程关注点
对话 LLM	单轮/多轮自然语言对话	不能真正操作环境	prompt 质量、上下文拼接
简单调用工具	调用外部工具	工具选择与结果整合不稳定	tool schema、调用约束、权限控制
循环多次调用工具	多步推理：根据结果多次调用工具	无任务状态管理、无错误重试机制、无中断/中止	重试机制，多步逻辑
多轮有状态与记忆管理	结构化状态和持久化记忆	长流程任务容易丢上下文、缺少阶段管理	状态&记忆管理

对话 LLM

概念定义与能力边界

基础 Chat LLM 对话的核心是：模型接受一组消息，输出一段文本回复。它可以表现出解释、问答、总结、改写、推理等能力，但本质上仍然停留在“语言空间”中。

它的能力边界通常是：

- 消息化上下文：相比“普通文本补全模型”，可以消息化上下文（system / user / assistant 均放到 messages 里，区分不同消息的角色类型），根据 messages 做语言生成，并且可以维护多轮对话一致性；不再只是“续写文本”，而是能以会话方式完成问答与协作；

- system prompt：嵌入角色、规则、格式要求。

- 模拟“计划”或“建议”：但模型仅被动响应，不能主动规划步骤；

- 不能与外部环境发生真实交互；就无法保证回答与外部世界状态一致（不能获取最新信息）；

因此，这一阶段更准确地说是“对话模型系统”，还不是严格意义上的工程化 Agent。

代码示例

```
import time
class MockLLM:
```



```

"""模拟 LLM 的回复，仅用于演示"""
def invoke(self, prompt: str) -> str:
    print(f"[LLM] 收到提示: {prompt[:50]}...")
    time.sleep(0.1) # 模拟网络延迟
    # 极简规则回复
    if "天气" in prompt:
        return "当前天气晴朗，气温 25°C。"
    elif "你好" in prompt:
        return "你好！有什么可以帮助你吗？"
    else:
        return "我不太理解你的问题，请换一种方式描述。"

def basic_chat():
    llm = MockLLM()
    print("=== 基础对话模型（无状态）===")
    while True:
        user_input = input("用户: ")
        if user_input.lower() in ["exit", "quit"]:
            break
        response = llm.invoke(user_input)
        print(f"AI: {response}")

if __name__ == "__main__":
    basic_chat()

```

这个示例的重点是：没有工具，没有环境，没有显式状态；可以说明“聊天模型”和“Agent”之间最初的边界。

简单调用工具 Agent

概念定义与能力边界

简单调用工具 agent 的关键变化是：模型不再只输出文本，也可以输出“我想调用哪个工具、传什么参数”。然后运行时去执行工具，再把结果返回给模型，让模型继续决定下一步或给出最终回复。

通过 Function Calling 机制，Agent 能够将用户的自然语言指令解析为对特定工具（函数）的调用。这时 Agent 才开始具备“行动能力”。

其能力边界通常是：

- 一次能调用一个工具，便可以访问环境，可以将文件读取、命令执行、数据库访问、检索等操作纳入推理链，把外部世界信息引入会话；

但它尚未解决：

- 不能基于工具结果继续推理，不能形成最基本的 Observe → Act → Observe 闭环。

代码示例

```

import json

from typing import Dict, Any, List

class MockToolLLM:
    """模拟支持工具调用的 LLM"""
    def invoke_with_tools(self, messages: List[Dict], tools: List[Dict]) -> Dict[str, Any]:
        user_msg = messages[-1]["content"]
        print(f"[LLM] 分析用户输入: {user_msg}")
        # 模拟决策: 检测关键词决定是否调用工具
        if "天气" in user_msg and "北京" in user_msg:
            # 所谓支持工具调用, 就是大模型返回调用函数的参数等信息
            return { "tool_calls": [{ "name": "get_weather", "arguments": {"city": "北京"}} ] }
        elif "计算" in user_msg: # 简单提取数字, 此处仅演示
            return { "tool_calls": [{ "name": "calculator", "arguments": {"expression": "3+5"}} ] }
        else:
            return {"content": "我是一个助手, 可以帮你查询天气或计算。"}

class ToolExecutor:
    """工具执行器 (Mock) """
    @staticmethod
    def get_weather(city: str) -> str:
        return f'{city} 当前晴朗, 22°C, 湿度 40%'
    @staticmethod
    def calculator(expression: str) -> str:
        # 仅用于演示, 实际应使用安全的 eval 或自定义解析
        try:
            result = eval(expression)
            return f'计算结果: {result}'
        except:
            return "计算表达式无效"
    def execute(self, tool_name: str, arguments: Dict) -> str:
        if tool_name == "get_weather":
            return self.get_weather(**arguments)
        elif tool_name == "calculator":
            return self.calculator(**arguments)
        else:
            return f'未知工具: {tool_name}'

def tool_agent_demo():
    llm = MockToolLLM()

```

```

executor = ToolExecutor()
tools = [ {"name": "get_weather", "description": "获取指定城市的天气",
"parameters": {"city": "string"}},
{"name": "calculator", "description": "执行数学计算", "parameters":
{"expression": "string"}} ]
print("=== Agent + 工具调用 ===")
user_input = input("用户: ")
messages = [{"role": "user", "content": user_input}]
# 第一次 LLM 调用
response = llm.invoke_with_tools(messages, tools)
if "tool_calls" in response:
    tool_call = response["tool_calls"][0]
    tool_name = tool_call["name"]
    args = tool_call["arguments"]
    print(f"[Agent] 决定调用工具: {tool_name}({args})")
    # 执行工具
    tool_result = executor.execute(tool_name, args)
    print(f"[工具返回]: {tool_result}")
    # 将工具结果作为新消息再次给 LLM
    messages.append({"role": "assistant", "tool_calls": [tool_call]})
    messages.append({"role": "tool", "content": tool_result})
    # 第二次 LLM 调用（生成最终回答）
    final_response = llm.invoke_with_tools(messages, tools)
    print(f"AI: {final_response.get('content', '操作完成')}")
else:
    print(f"AI: {response.get('content')}")

if name == "main":
    tool_agent_demo()

```

这个示例刻意保持简单，但已经体现了第二阶段的本质：

- 模型可以输出 ToolCall，- Tool 是结构化接口，可 mock；
- 运行时把工具结果再喂给模型；

循环调用工具

概念定义与能力边界

能做什么：

多步推理：模型可多次调用工具，每次根据结果决定下一步（观察→行动→再观察）。

简单条件分支：例如先查天气，温度低则调用穿衣建议。

不能做什么：

- ✗ 无任务状态管理：无法记忆累加器、阶段标志，容易“忘记”中间状态。
- ✗ 无错误重试机制：工具失败时可能乱试或放弃。

✗ 无中断/中止：可能死循环或超时，只能硬截断。

代码示例

```
# 1. 定义一个工具（计算器）
```

```
def calculator(expr: str) -> str:
```

```
    try:
```

```
        return str(eval(expr))
```

```
    except:
```

```
        return "计算错误"
```

```
# 2. Mock 的 LLM 决策函数（根据对话历史模拟思考）
```

```
def mock_llm(messages):
```

```
    # 检查最后几条消息，决定下一步动作
```

```
    history_str = str(messages)
```

```
    # 需要两次调用：先算 1+2，再算 3*3
```

```
    if "1+2" in history_str and "calculator" in history_str and "3*3" not in history_str:
```

```
        # 已经有 1+2 的结果，接下来调用计算 3*3
```

```
        return {"type": "tool_call", "name": "calculator", "args": {"expr": "3*3"}}
```

```
    elif "1+2" in history_str and "calculator" not in history_str:
```

```
        # 第一次调用：计算 1+2
```

```
        return {"type": "tool_call", "name": "calculator", "args": {"expr": "1+2"}}
```

```
else:
```

```
# 工具都调用完了，返回最终答案
```

```
return {"type": "answer", "content": f'最终计算结果: {messages[-1][\'content\']}'}"
```

```
# 3. Agent 主循环
```

```
def run_agent(user_query):
```

```
    messages = [{"role": "user", "content": user_query}]
```

```
    while True:
```

```
        decision = mock_llm(messages)
```

```
        if decision["type"] == "answer":
```

```
            print(f'最终回答: {decision[\'content\']}')"
```

```
            return decision["content"]
```

```
# 执行工具调用
```

```
    tool_name = decision["name"]
```

```
    args = decision["args"]
```

```
    result = calculator(**args)
```

```
    print(f'[调用工具] {tool_name}({args}) -> {result}')
```

```
# 将工具结果加入对话历史
```

```
messages.append({"role": "ai", "content": decision})
```

```
messages.append({"role": "tool", "content": result})
```

```
# 4. 运行
```

```
if __name__ == "__main__":
```

```
run_agent("请计算 (1+2) 的结果，然后将结果乘以 3")
```

有状态/记忆管理 Agent

概念定义与能力边界

有状态/记忆 agent 的关键变化是：Agent 不再只依赖 messages，而是拥有独立、结构化、可持久化的状态；同时开始引入“记忆”概念，并且不只讨论“存了什么”，而要讨论“怎么管理”。

这是从“会用工具的对话循环”走向“任务执行系统”的分水岭。相较简单 Tool-calling Agent，多轮有状态 Agent 新增了：

- 显式任务状态，而不再只靠 messages 猜当前阶段，让模型对“当前做到哪一步”有稳定认识；

- 中断后恢复；

- 对不同阶段应用不同规则；

- 将错误、补丁、经验记录为结构化数据，把经验系统化积累；

- 跨轮次、跨会话保存状态和记忆。

代码示例

```
class MockMemoryAgent:
```

```
    """
```

```
    模拟一个具有记忆和多轮状态的智能体。
```

- 记忆：保存所有交互历史（用户输入和系统输出）

- 状态：维护一个可自定义的状态字典，用于跨轮次跟踪信息

```
    """
```



```

def init (self, initial_state=None):
    # 记忆列表：存储每一轮的（用户输入，系统输出）
    self.memory = []
    # 状态字典：用于存储多轮状态变量（如计数器、标志、用户偏好等）
    self.state = initial_state if initial_state is not None else {}
    # 可选：自定义响应生成函数，默认使用简单的规则
    self.response_generator = self.default_response

def default_response(self, user_input):
    """默认响应生成器：基于输入内容和当前状态返回模拟响应"""
    # 示例状态：记录用户问候次数
    if 'greeting_count' not in self.state:
        self.state['greeting_count'] = 0
    # 简单的模式匹配
    user_lower = user_input.lower()
    if '你好' in user_lower or 'hello' in user_lower:
        self.state['greeting_count'] += 1
        if self.state['greeting_count'] == 1:
            return "你好！很高兴见到你。"
        else:
            return f"我们又见面了！这是第 {self.state['greeting_count']} 次问候。"
    elif '天气' in user_lower:
        return "今天天气晴朗，适合出门。"
    elif '状态' in user_lower:
        # 展示当前状态（演示用）
        return f"当前状态: {self.state}"
    elif '重置' in user_lower:
        # 重置所有状态（记忆不清除，但状态重置）
        self.state = {}
        return "状态已重置。"
    elif '记忆' in user_lower:
        # 返回最近三条记忆
        recent = self.memory[-3:] if self.memory else []
        return f"最近的记忆: {recent}"
    elif '退出' in user_lower:
        return "再见！"
    else:
        return f"我不太理解 '{user_input}'，请再试一次。"

def process_input(self, user_input):
    """处理用户输入：更新记忆，调用响应生成器，返回响应"""
    # 调用响应生成器获取回复
    response = self.response_generator(user_input)
    # 将本次交互存入记忆
    self.memory.append((user_input, response))

```

```

        return response
# 示例使用
if __name__ == "__main__":
    agent = MockMemoryAgent(initial_state={"step": 0})
    # 模拟多轮对话
    inputs = [
        "你好",
        "今天天气怎么样",
        "你好",
        "状态",
        "你好",
        "记忆",
        "重置",
        "状态"
    ]
    for user_input in inputs:
        response = agent.process_input(user_input)
        print(f"用户: {user_input}")
        print(f"Agent: {response}\n")

```

代码说明：

`memory` 列表保存每一轮的（用户输入，系统输出）对，实现了简单的记忆机制。

`state` 字典用于存储任意多轮状态（如计数器、会话阶段、用户信息等）。

默认响应生成器演示了如何利用 `state` 实现带记忆的逻辑（例如统计问候次数）。

示例运行展示了多轮交互，包括状态查询、记忆查询、状态重置等功能。

运行示例代码后，输出类似：

用户：你好 Agent：你好！很高兴见到你。用户：今天天气怎么样 Agent：今天天气晴朗，适合出门。用户：你好 Agent：我们又见面了！这是第 2 次问候。用户：状态 Agent：当前状态：{'greeting_count': 2, 'step': 0} 用户：你好 Agent：我们又见面了！这是第 3 次问候。用户：记忆 Agent：最近的记忆：[('你好', '我们又见面了！这是第 2 次问候。'), ('今天天气怎么样', '今天天气晴朗，适合出门。'), ('你好', '我们又见面了！这是第 3 次问候。')] 用户：重置 Agent：状态已重置。用户：状态 Agent：当前状态：{}

Agent 实践 改进方向

更强的工具治理：能力标签、风险分级、预算控制

- 给每个 tool 增加 capability tags，如 `read\_only`、`write\_file`、`external\_api`；
- 增加风险分级，如 low / medium / high；
- 增加 tool audit trail，用于后续回放与评估；
- 为高风险工具引入额外审批或二次确认。

自反思 evaluator / reflection middleware

- 对每轮输出做 evaluator 评分；
- 判断当前结果是否满足退出条件；
- 判断是否需要重试、切换策略、或者转人工；
- 将评估结果写入结构化状态。

多 Agent 编排：planner / executor / verifier / memory curator

- planner：制定执行计划；
- executor：执行工具与具体操作；
- verifier：检查结果是否达标；
- memory curator：整理和筛选可进入长期记忆的经验。

参考资料

Hello-agents